

UNIVERSIDAD DE BUENOS AIRES
FACULTAD DE CIENCIAS EXACTAS Y NATURALES
DEPARTAMENTO DE COMPUTACIÓN



BASES DE DATOS
No Relacionales

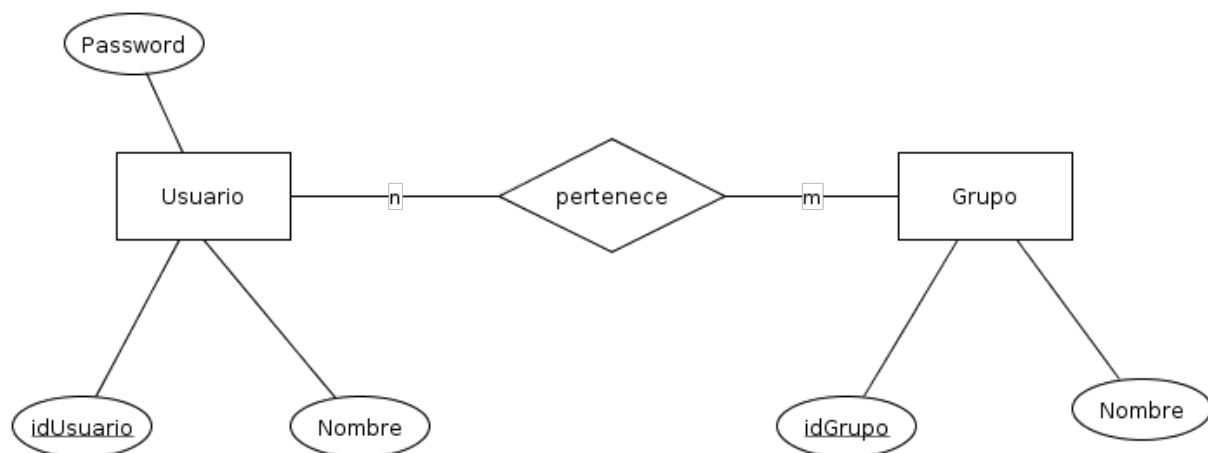
GUÍA DE EJERCICIOS

1 Conceptuales

- 1.1. Describa brevemente limitaciones de las base de datos relacionales y con qué características las base de datos NoSql mitigan este problema.
- 1.2. Describa los cuatro tipos de base de datos NoSql.
- 1.3. ¿Qué es un espacio de nombres o bucket en una base de datos *Key-Value*?
- 1.4. ¿De ejemplos de uso de TTL (time to live) en una base *Key-Value*?
- 1.5. Compare *Key-Value* con *Document Database*, de ventajas y desventajas de una u otra.
- 1.6. Discuta ventajas y desventajas de que una *Document Database* sea *schemaless*.
- 1.7. ¿En qué casos puede ser conveniente *desnormalizar*?
- 1.8. Compare las *Column Family Databases* con otros tipos de bases de datos NoSQL.
- 1.9. ¿A que se denomina consistencia eventual?
- 1.10. Explique el teorema CAP.

2 Modelización y Diseño

- 2.1. Dado el siguiente DER realizar por lo menos 3 diferentes esquemas para *Document Databases*.



- 2.2. Dado el siguiente ejemplo en JSON sobre editoriales libros

```

{
  "titulo": "MongoDB: The Definitive Guide",
  "autor": [ "Kristina Chodorow", "Mike Dirolf" ],
  "fecha_pub": ISODate("2010-09-24"),
  "paginas": 216,
  "idioma": "English",
  "editorial": {
    "nombre": "O'Reilly Media",
  }
}
  
```

```

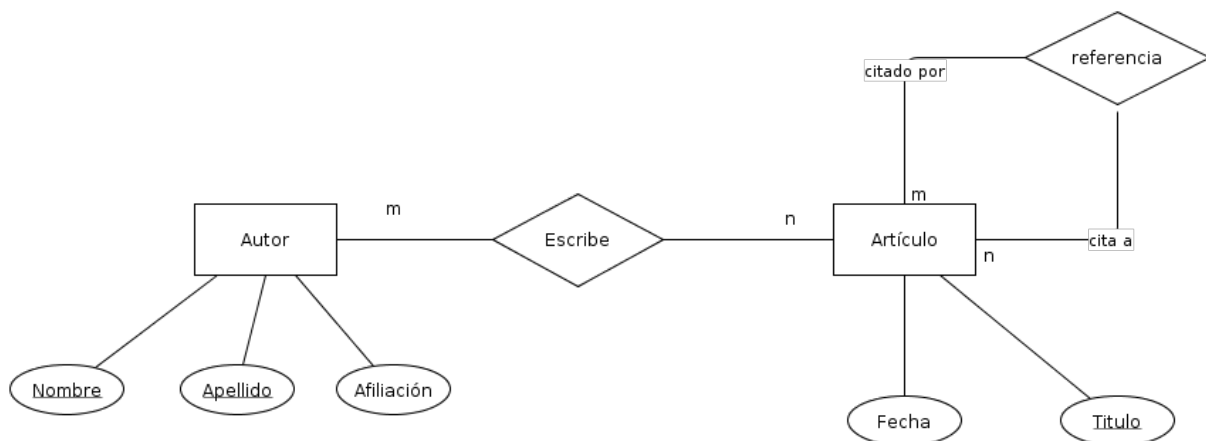
    "fundada": 1980,
    "ubicacion": "CA"
  }
}

{
  "titulo": "50 Tips and Tricks for MongoDB Developer",
  "autor": "Kristina Chodorow",
  "fecha_pub": ISODate("2011-05-06"),
  "paginas": 68,
  "idioma": "English",
  "editorial": {
    "nombre": "O'Reilly Media",
    "fundada": 1980,
    "ubicacion": "CA"
  }
}

```

- (a) Modificar los documentos para que no se repitan los datos de las editoriales
- (b) Discutir en que contexto es mejor la representación dada sobre la resuelta en el ítem anterior.

2.3. Un congreso de ciencias de la computación almacena datos de las publicaciones realizadas. Se guardan autores, artículos y además las relaciones entre artículos (es decir artículos que citan otros artículos). El siguiente DER modela la situación.



Ej. 2.3

Se pide que realice un modelo para una base de datos *Column-Family* tal que responda las siguientes consultas.

- (a) Buscar todos los artículos que citan a un artículo dado. Se debe devolver fecha y título de los artículos citados.
- (b) Buscar todos los autores que escribieron en una fecha dada.

2.4. Una empresa de video juegos realiza un juego en línea y necesita guardar el estado de las partidas de los jugadores. Dicho estado debe almacenar: posición, nivel de vida, objetos encontrados y enemigos abatidos. El jugador deberá poder jugar desde cualquier estado

guardado eligiendo la fecha y hora en el que lo guardo. Se pide realizar un modelo para una base *Key-Value* que soporte lo descrito.

- 2.5. Una aplicación desea guardar información de sus usuarios, los mismos son identificados por su dirección de correo. Podemos asumir dos tamaños de la información guardar pequeño y grande. La información de tamaño pequeño es:

- Nombre
- Genero
- Dirección
- Teléfono

La información de tamaño grande incluye:

- Imagen
- Clave pública 1
- Clave pública 2
- Mensaje de voz de saludos

Diseñar para una base de datos *Key-Value* la forma de organizar la clave para los siguientes casos:

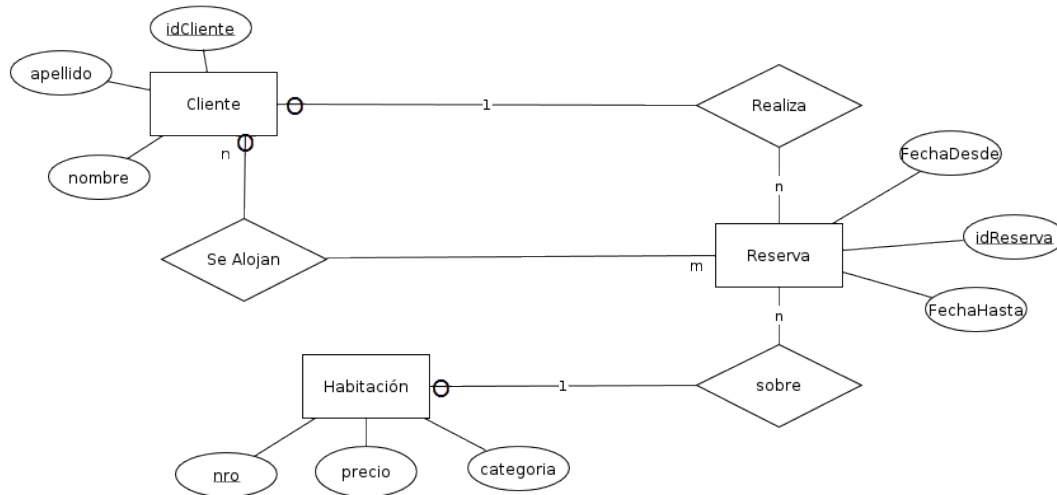
Las forma de acceso es:

- (a) Las propiedades de tamaño pequeño son accedidas generalmente en conjunto.
- (b) Las propiedades de tamaño grande son leídas cuando un usuario busca su información
- (c) Al actualizar los datos las claves públicas son siempre actualizadas al mismo tiempo
- (d) Tanto la imagen como el mensaje de voz puede ser actualizados en forma independiente de las otras cosas.

- 2.6. Una empresa provee acceso mediante tarjetas con lectores. Cada tarjeta tiene un identificador que es leído por los lectores para saber si tiene acceso en cada lugar. Cada puerta esta identificada con un número y cuenta con un lector. Si el identificador de tarjeta tiene permiso la puerta se abre y sino no. Para saber si un identificador tiene permiso los mismos se agrupan en roles. Los roles son los que tienen el permiso de acceso. Un identificador de tarjeta puede pertenecer a varios roles. Una vez que una persona entra en la recepción se le da una tarjeta y se activa la misma, cuando se retira de la empresa la devuelve en recepción y se desactiva.

Se pide:

- (a) Realizar un diseño para mantener el recorrido las tarjetas activas en una base clave-valor. Es decir debe guardarse la tarjeta con las puertas por las que paso y por las que intento pasar. También debe saberse en que momento abrió o intento abrir una puerta. Además la base debe poder usarse para determinar si una tarjeta puede o no acceder a una puerta.
- (b) Realizar un diseño basado en documentos para modelar el problema. Debe realizar primero un **modelo conceptual** y posteriormente el **diagrama de interacción de documentos**. Explicar fundamentadamente las decisiones de diseño tomadas. Cuando la persona que tiene la tarjeta se retira toda la información asociada a la tarjeta que está en la base clave valor (según ítem 2.6.a) debe pasarse a la base de documentos y es eliminada de la base clave valor. Tome en cuenta que esa información es enormemente grande a lo largo del tiempo. No debería perder información al modelar los documentos.



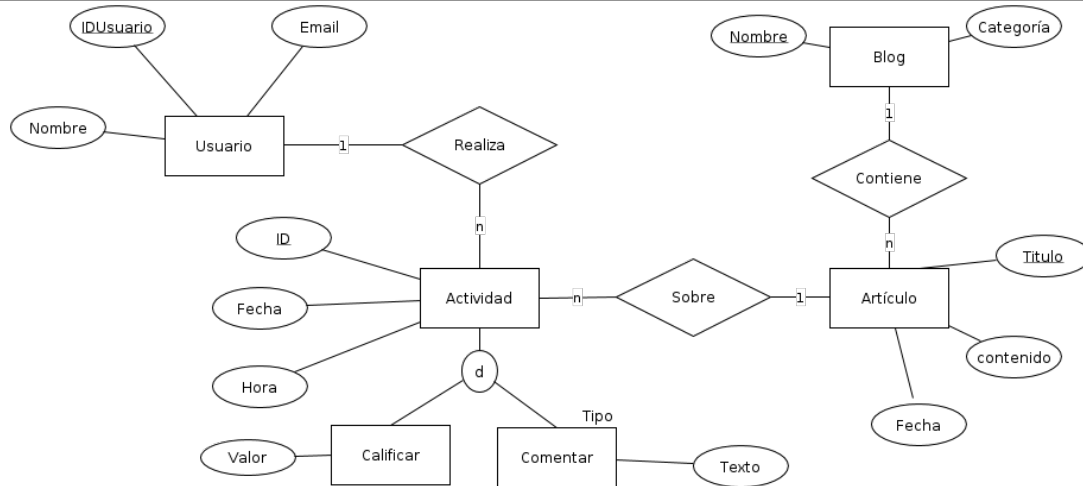
Ej. 2.7

2.7. Dado el DER de la figura , donde se modela la reserva de habitaciones en un hotel coherentes con la funcionalidad descrita.

- Se pide que se realice un modelo completo del mismo para una base de datos *Document Based*. El modelo **debe** incluir diagrama de interrelación de documentos y JSONSchema completo del diseño. Tome en cuenta lo siguiente: Un cliente realiza una reserva para una habitación en un rango de fechas. La reserva es sobre una habitación y para una cantidad de personas (son los que se alojan y también son clientes). El precio que figura en habitación es el precio por noche. Cuando se consulta una reserva se necesita conocer el detalle de la habitación reservada y los datos del cliente que hizo la reserva y eventualmente la lista de pasajeros. El precio de la habitación puede cambiar con el tiempo por lo que se necesita saber a que precio se reservó la habitación es su momento. Debe fundamentar brevemente las elecciones de diseño que realice y deben ser
- Se pide que realice el diseño (utilizando el diagrama Chebotko) para una base de datos *Column-Family* tal que responda la siguiente consulta:
Los datos de las habitaciones que para una fecha dada comienzan una reserva en esa fecha ordenadas por precio descendente.
- Diseñe una base clave-valor para poder consultar si hay alguna habitación de una categoría dada reservada en una fecha. Es decir se debe saber las habitaciones de una categoría y si esa habitación esta reservada en una fecha dada.

2.8. Dado el siguiente DER, donde se modela un blog con artículos y acciones de usuarios sobre ellos (calificar o comentar)

- Se pide que se realice un modelo para una base de datos *Document Based*. Tome en cuenta que:
 - Se quiere *particionar* las actividades por Fecha para cada usuario, es decir acceder en una única consulta para un usuario y fecha dadas a todas las actividades correspondientes.



Ej. 2.8

- ii. Para el caso de los artículos se quiere mostrar todas las actividades sobre él cada vez que se accede al mismo independientemente de la fecha.

Realizar el *JSONSchema* correspondiente.

- (b) Se pide que realice el diseño de las tablas para una base de datos de Column-Family tal que responda las siguientes consultas:
- Todos los artículos (título y contenido) de un Blog dado para una Fecha dada.
 - Todos los comentarios de un usuario en un rango de fechas.

2.9. El DER de la figura muestra un modelo de datos conceptual para un sistema de un club de cocina.

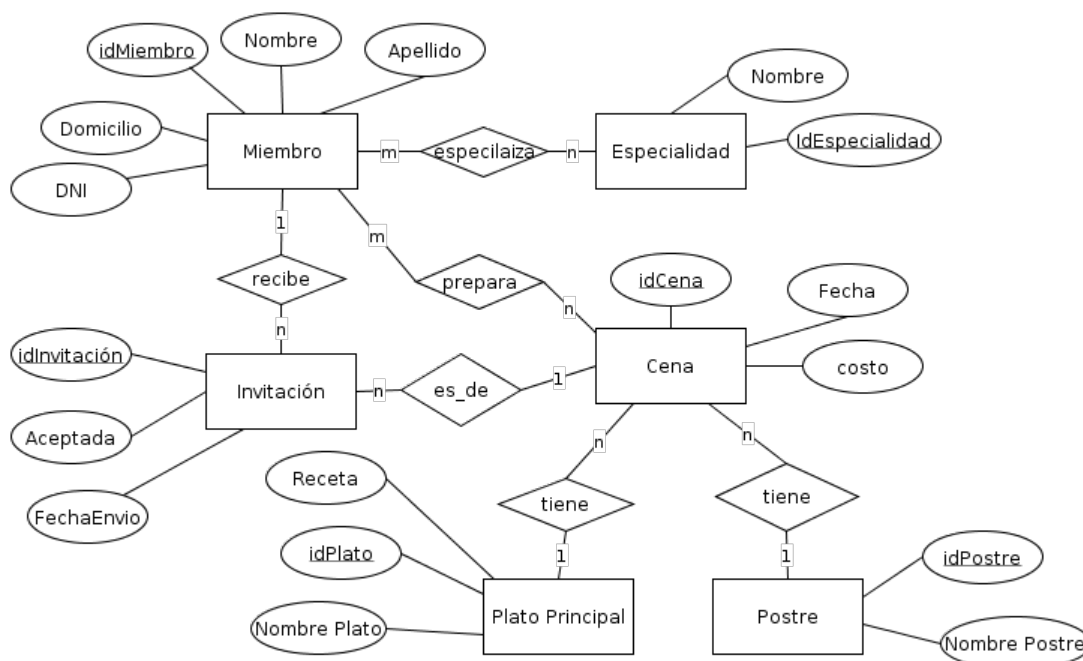


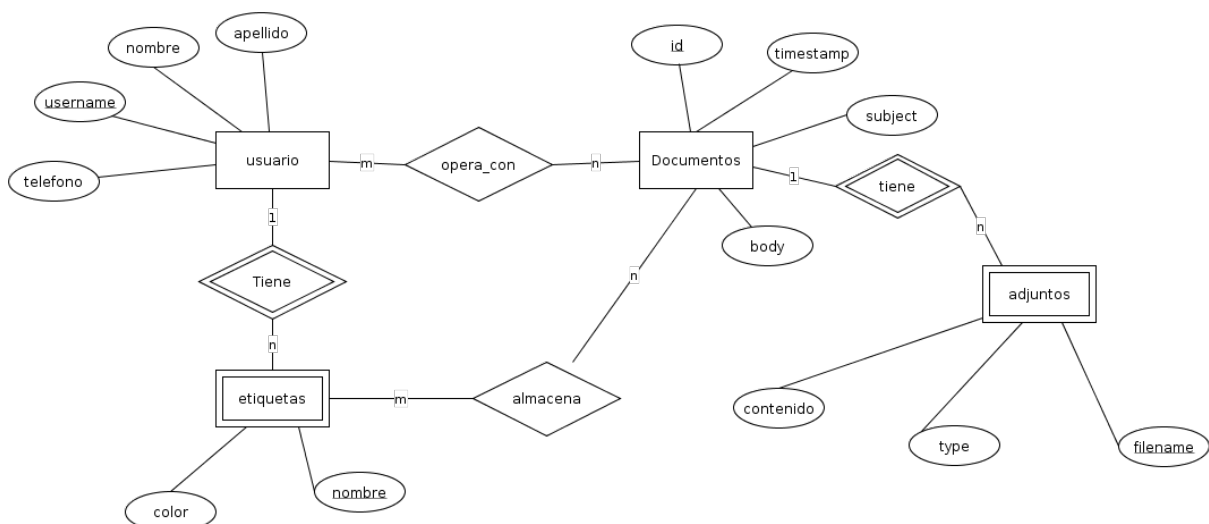
Figure 1: Ej.2.9

Los miembros del club pueden preparar cenas e invitar a otros miembros. Cada cena se basa en un plato único y un solo postre. Este plato principal y postre se pueden usar

nuevamente para otras cenas. Es importante conocer quienes preparan la cena y quienes asisten a la misma. Al organizar la cena se genera una invitación que es enviada a cada miembro del club e incluye la fecha de la cena, el costo y la ubicación de la misma. Los miembros que van a participar deben aceptar la invitación. Los miembros del club tienen una o varias especialidades de cocina

- (a) Realizar el modelo lógico de interrelación de documentos correspondiente al DER incluido y un JSONSchema de algún tipo de documento con al menos un nivel de anidamiento. Tomar en cuenta los siguientes ítems::
- La impresión de invitaciones es una de las tareas más frecuentes. En las mismas se imprime toda a info de la cena incluyendo los nombres de platos/postres.
 - Ocasionalmente se consultan miembros de alguna especialidad, pero no es algo que se haga muy seguido.
 - Es frecuente consultar las fechas en las cuales un miembro organizó cenas
- (b) Realizar el diagrama de *Chebotko*, justificando cada paso, para una base **Column-Family** que responda a las siguientes consultas:
- Nombres y Apellidos de los miembros invitados a una cena dada.
 - Todas las cenas preparadas por un miembro dado desde el año 2015, ordenadas por costo de menor a mayor.
- (c) Realizar el diseño para una base de datos **Key-Value** que permita guardar y consultar la información de miembros (según DER). La aplicación que usaría la base de datos ingresa con el DNI del miembro y una password. Inmediatamente ve sus datos y su especialidad. Luego tiene dos opciones: consultar sus invitaciones o las cenas que preparó.

2.10. Dado el DER de la figura que modela conceptualmente los datos de usuarios y documentos. Los usuarios operan con documentos compartidos, cada usuario puede asignar etiquetas propias para organizar sus documentos (solo visibles para él). A su vez los documentos pueden tener uno o más adjuntos.



Ej.2.10

- (a) Realizar el modelo lógico de interrelación de documentos correspondiente al DER incluido un JSONSchema con al menos un nivel de anidamiento. Tomar en cuenta los siguientes ítems:

- Cuando el usuario entra al sistema debe poder ver rápidamente una lista de los documentos con los que opera (subject, fecha y etiquetas asignadas a ellos por él) ordenados por fecha.
 - Al acceder a un documento es importante contar con todos los adjuntos disponibles
 - Al visualizar un documento debe poder verse los nombres de los usuarios que operan con él.
- (b) Realizar el diagrama de *Chebotko*, justificando cada paso, para una base **Column-Family** que responda a la siguiente consulta:
- i. Todos los documentos de un usuario ordenados ascendentemente por timestamp.
 - ii. Los nombres, apellidos y teléfonos de los usuarios que operan documentos generados en una fecha dada.
- (c) Realice un diseño **Key-Value** que permita guardar usuarios y documentos con sus etiquetas. Para los adjuntos sólo se necesita guardar el filename y el type. Considerar el username como inmutable.